

## Placement Tracker (Java / Spring Boot) — Knowledge Bytes

*Read-only analysis of D:\PROJECTS\Placement Tracker - Java\placement-tracker. This project is server-rendered Spring MVC + Thymeleaf (no REST API, no separate JS frontend). Each byte covers ONE concept. Order is progressive: architecture first, then layers, then details and edge cases.*

---

### Byte 1: Big picture: how this project actually works

**Builds on:** None — starting point

#### In plain terms:

This is a browser form app, not a phone-style REST + JavaScript app. The browser talks to the Java server with plain page loads and HTML form posts, and the server replies with fully rendered HTML pages.

#### The code:

```
[text] (from this project – trimmed to the relevant lines)
Browser -> Controller -> Service -> Repository -> Hibernate -> SQLite (placement_tracker.db)
-> Repository -> Service -> Controller -> Thymeleaf template -> HTML -> Browser
```

#### What's happening:

Every user action follows the same loop. The browser sends an HTTP request (GET to view a page, POST to submit a form). ApplicationController handles web concerns, ApplicationService enforces business rules, ApplicationRepository (Spring Data JPA + Hibernate) reads/writes the SQLite file, and a Thymeleaf template turns the result back into HTML. There is no JSON API and no frontend fetch() layer in this project.

#### Why it matters:

If you expect a REST API you will look in the wrong place. Knowing this is server-rendered MVC explains why controllers return template names like "index" instead of JSON.

---

### Byte 2: Application startup: the one entry point

**Builds on:** Byte 1

#### In plain terms:

One tiny class boots the whole server. It switches on embedded Tomcat, Spring MVC, Thymeleaf, and JPA automatically based on the libraries in pom.xml.

#### The code:

```
[java] (from this project – trimmed to the relevant lines)
@SpringBootApplication
public class PlacementTrackerApplication {
    public static void main(String[] args) {
        SpringApplication.run(PlacementTrackerApplication.class, args);
    }
}
```

### What's happening:

@SpringBootApplication is three things at once: configuration, auto-configuration, and component scanning. At startup Spring scans `com.example.placementtracker`, finds @Controller, @Service, @Repository, and @Component classes, and wires them together. SpringApplication.run starts the embedded Tomcat on port 8080.

### Why it matters:

Without this class nothing runs. Every other byte only executes because this boot step discovered and connected the components.

---

## Byte 3: Dependencies: what each library is for (pom.xml)

**Builds on:** Byte 2

### In plain terms:

The pom.xml lists six ingredients. Each one unlocks one layer of the app: web pages, HTML templates, database mapping, validation, the SQLite file, and tests.

### The code:

```
[xml] (from this project – trimmed to the relevant lines)
spring-boot-starter-web           // MVC (@Controller, @GetMapping) + Tomcat
spring-boot-starter-thymeleaf     // dynamic HTML rendering
spring-boot-starter-data-jpa      // repositories + Hibernate ORM
spring-boot-starter-validation    // @NotBlank, @Size checks
sqlite-jdbc                       // SQLite file driver

hibernate-community-dialects      // lets Hibernate generate SQLite SQL
```

### What's happening:

Spring Boot 3.2.5 (Java 17) manages compatible versions. spring-boot-starter-web brings Spring MVC. Thymeleaf renders the pages. spring-boot-starter-data-jpa brings JpaRepository and Hibernate. Validation brings Jakarta Bean Validation. The last two are SQLite-specific: the JDBC driver plus the Hibernate dialect that translates generic operations into SQLite SQL.

### Why it matters:

If a layer confuses you, check this list first. For example, th:\* attributes in HTML only work because the Thymeleaf starter is present.

---

#### Byte 4: Configuration: connecting code to the SQLite file (application.properties)

**Builds on:** Byte 3

##### **In plain terms:**

A 14-line config file points the app at a real database file and tells Hibernate how to behave. Delete the file and you get a fresh database on next run.

##### **The code:**

```
[properties] (from this project – trimmed to the relevant lines)
server.port=8080
spring.datasource.url=jdbc:sqlite:placement_tracker.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.thymeleaf.cache=false
```

##### **What's happening:**

server.port fixes the URL at http://localhost:8080. The datasource URL is a relative file path, so placement\_tracker.db is created in the project folder and survives restarts. ddl-auto=update makes Hibernate create/alter the applications table from the @Entity automatically — no SQL scripts. show-sql=false hides SQL logs. thymeleaf.cache=false lets you edit HTML and just refresh during development.

##### **Why it matters:**

This is why there is no database setup step. The file location, the dialect, and auto-DDL together explain persistence without any SQL files in the project.

---

#### Byte 5: The model: one Application object = one database row

**Builds on:** Byte 1

##### **In plain terms:**

The Application class is both the form object and the database row. Each field becomes a column in the applications table.

##### **The code:**

```
[java] (from this project – trimmed to the relevant lines)
@Entity
@Table(name = "applications")
public class Application {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

    private String company;
    private String role;
    private String status;
    private LocalDate appliedOn;
    // + jobUrl, notes, createdAt (+ getters/setters)
}

```

### What's happening:

@Entity tells Hibernate to persist this class. @Table names the table applications. @Id plus IDENTITY makes id an auto-increment primary key (SQLite rowid). The remaining fields map to columns. Getters/setters are not boilerplate here — JPA, Thymeleaf, and form binding all call them by reflection.

### Why it matters:

This single class is the shared language of every layer: the HTML form binds to it, the service validates it, and Hibernate stores it.

---

## Byte 6: Field rules part 1: simple validation with annotations

**Builds on:** Byte 5

### In plain terms:

Short rules like required and max length sit directly on the fields. They run automatically whenever a form is submitted.

### The code:

```

[java] (from this project – trimmed to the relevant lines)
@NotBlank(message = "Company is required")
@Size(min = 2, max = 100, message = "Company must be 2-100 characters")
@Column(nullable = false, length = 100)
private String company;

@Size(max = 500, message = "Job URL must be under 500 characters")
@Column(name = "job_url", length = 500)
private String jobUrl;

```

### What's happening:

@NotBlank rejects null/empty/blank strings; @Size checks length. @Column mirrors part of that at the database level (nullable=false, length limits, custom column names like job\_url and applied\_on, TEXT for notes). @DateTimeFormat(iso = ISO.DATE) on appliedOn parses the HTML date input (yyyy-MM-dd) into a LocalDate. status is deliberately a String, not a Java enum (see Byte 7).

### Why it matters:

These annotations catch typos and empty fields before any business logic or SQL runs, and their messages are what the form redisplay beside the offending field.

---

## Byte 7: Why status is a String and not an Enum

**Builds on:** Byte 6

### In plain terms:

Status looks like it should be an enum (fixed set of values), but the project keeps it a plain String on purpose so bad input shows a friendly form error instead of crashing.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
// Kept as String (not Enum) so the HTML form binds easily
// and wrong values can show a friendly error instead of crashing.
@NotBlank(message = "Status is required")
@Column(nullable = false, length = 20)
private String status;
```

### What's happening:

If status were an enum, a tampered form value (e.g. status=Hired) would fail type conversion before validation and produce a generic error page. As a String, binding always succeeds, and then the explicit whitelist check in the service/controller can attach a readable message to the status field.

### Why it matters:

A one-line type choice controls the whole error UX. It trades compile-time safety for graceful handling of user-tampered input.

---

## Byte 8: Auto timestamp: @PrePersist fills createdAt

**Builds on:** Byte 5

### In plain terms:

The app never asks the user for a creation date. The entity stamps it itself the first time it is saved.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
@Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@PrePersist
public void prePersist() {
    if (createdAt == null) {
        createdAt = LocalDateTime.now();
    }
}
```

### What's happening:

@PrePersist marks a callback Hibernate invokes just before an INSERT. updatable=false means later updates never overwrite it. The no-args constructor exists because JPA needs it to rehydrate rows into objects. createdAt is also the sort key for the dashboard (newest first).

#### **Why it matters:**

Without this, every create path would have to remember to set the timestamp. Centralising it guarantees ordering and audit info for free.

---

### **Byte 9: The repository: full CRUD with zero SQL**

**Builds on:** Byte 5

#### **In plain terms:**

The entire database-access layer is one empty interface. Spring writes the implementation for you at startup.

#### **The code:**

```
[java] (from this project – trimmed to the relevant lines)
@Repository
public interface ApplicationRepository extends JpaRepository<Application, Long> {
}
```

#### **What's happening:**

By extending JpaRepository<Application, Long>, this interface inherits save, findAll(Sort), findById, count, and deleteById. Spring Data JPA generates the implementation; Hibernate turns each call into SQLite SQL using the SQLite dialect. No method bodies and no SQL strings exist in the project — and that is intentional.

#### **Why it matters:**

Every database operation in the service (Bytes 11-13) is just a call on this interface. If you are hunting for SQL, stop: there is none to find.

---

### **Byte 10: The service: where the real business rules live**

**Builds on:** Bytes 5, 9

#### **In plain terms:**

The service sits between the web layer and the database. It owns the two rules that simple annotations cannot express: which statuses are legal, and when the applied date is required.

#### **The code:**

```
[java] (from this project – trimmed to the relevant lines)
@Service
public class ApplicationService {
    public static final List<String> ALLOWED_STATUSES =
        List.of("Wishlist", "Applied", "Interview", "Offer", "Rejected");
    private final ApplicationRepository repository;
    public ApplicationService(ApplicationRepository repository) {
        this.repository = repository; // constructor injection
    }
}
```

#### What's happening:

@Service marks this as business logic. ALLOWED\_STATUSES is public and static so both the service and the controller/templates can share the exact same dropdown values. Constructor injection (no @Autowired field) gives the service its repository; Spring passes it automatically and tests can pass a mock instead.

#### Why it matters:

Keeping rules here (not in the controller) means create and update share one enforcement point, and the dropdown can never drift out of sync with validation.

---

### Byte 11: Service reads: findAll sorted + findByIdOrThrow

**Builds on:** Byte 10

#### In plain terms:

Two read methods power the dashboard and every page that needs one record. One sorts newest-first; the other converts a missing row into a 404 exception.

#### The code:

```
[java] (from this project – trimmed to the relevant lines)
public List<Application> findAll() {
    return repository.findAll(Sort.by(Sort.Direction.DESC, "createdAt"));
}
public Application findByIdOrThrow(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Application not found: " + id));
}
```

#### What's happening:

findAll delegates to the repository with an explicit sort on createdAt descending, so the dashboard shows recent entries first. findById returns an Optional; orElseThrow unwraps it or throws the project's custom unchecked exception carrying the missing id. The controller maps that exception to the 404 page (Byte 20).

#### Why it matters:

Sorting here (not in the template) keeps display order consistent everywhere, and throwing instead of returning null forces every caller to handle the missing-record case.

---

### Byte 12: Service writes: create, update, deleteById

**Builds on:** Bytes 10, 11

#### In plain terms:

Writes all funnel through validation first. Update loads the stored row and copies fields into it rather than saving the form object directly.

#### The code:

```
[java] (from this project – trimmed to the relevant lines)
public Application create(Application app) {
    validate(app);
    return repository.save(app);          // INSERT
}
public Application update(Long id, Application updated) {
    Application existing = findByIdOrThrow(id);
    existing.setCompany(updated.getCompany());
    // ... role, status, appliedOn, jobUrl, notes ...
    validate(existing);
    return repository.save(existing);    // UPDATE
}
public void deleteById(Long id) {
    findByIdOrThrow(id);                // 404 if missing
    repository.deleteById(id);
}
```

#### What's happening:

create validates then save() inserts (id is null, createdAt auto-filled). update first re-loads the managed entity — preserving its id and createdAt — copies each editable field from the form object, validates, then save() issues an UPDATE. delete checks existence first so deleting a stale id still yields a 404 instead of silently succeeding.

#### Why it matters:

Copy-onto-managed-entity is the safest update pattern: it prevents over-posting the id/createdAt and guarantees validation runs on the exact state being stored.

---

### Byte 13: Service validation: the two rules annotations cannot do

**Builds on:** Byte 12

#### In plain terms:

Two cross-field rules live in one `validate()` method called by both `create` and `update`. Break either and you get an `IllegalArgumentException` with a user-readable message.

**The code:**

```
[java] (from this project – trimmed to the relevant lines)
public void validate(Application app) {
    if (app.getStatus() == null || !ALLOWED_STATUSES.contains(app.getStatus())) {
        throw new IllegalArgumentException(
            "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected");
    }
    if (!"Wishlist".equals(app.getStatus()) && app.getAppliedOn() == null) {
        throw new IllegalArgumentException(
            "Applied date is required unless status is Wishlist");
    }
}
```

**What's happening:**

Rule 1 is a whitelist: status must be exactly one of the five strings. Rule 2 is conditional: `appliedOn` may be null only when status is `Wishlist` (idea: you have not applied yet). `@NotBlank/@Size` cannot express either, so plain Java does it. The controller mirrors this logic pre-save to pin errors to the right field, and also catches this exception as a backstop (Byte 19).

**Why it matters:**

This is the domain heart of the app. Everything else is plumbing; this method is the rule a placement officer would actually state out loud.

---

**Byte 14: The 404 exception: a tiny class with one job**

**Builds on:** Byte 11

**In plain terms:**

A five-line unchecked exception carries the missing id from the service to the controller's 404 handler. No status code lives here — the controller adds that.

**The code:**

```
[java] (from this project – trimmed to the relevant lines)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

**What's happening:**

Extending `RuntimeException` means callers do not need `try/catch`; it propagates up to the `@ExceptionHandler`. The message (Application not found: 999) is displayed on the custom 404 page. It is in the service package because only the service throws it.

### Why it matters:

A dedicated type lets the controller catch only missing-record cases, leaving real bugs (e.g. DB down) to surface differently instead of masquerading as 404s.

---

### Byte 15: The controller: the frontend's only doorway (@Controller, not @RestController)

**Builds on:** Bytes 10, 14

### In plain terms:

One @Controller class owns all seven browser routes. Each method takes HTTP in, calls the service, and picks which HTML page comes back. It never returns JSON.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
@Controller
public class ApplicationController {
    private final ApplicationService service;
    public ApplicationController(ApplicationService service) {
        this.service = service;
    }
}
```

### What's happening:

@Controller (not @RestController) means return values are Thymeleaf template names (index, application-form, application-detail, 404) or redirect: strings — not response bodies. Constructor injection supplies the service. Note what is absent: no @RequestBody, no ResponseEntity, no DTOs, no security annotations — the project binds forms directly to the entity and has no login.

### Why it matters:

This distinction answers the frontend question: there is no JSON contract to consume. The frontend IS the templates in Bytes 23-27, driven by these return values.

---

### Byte 16: Route 1+2: dashboard and blank create form (GET)

**Builds on:** Byte 15

### In plain terms:

Two GET routes render read-only pages: the dashboard table and the empty Add form with Applied pre-selected.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
@GetMapping("/")
public String dashboard(Model model) {
```

```

        model.addAttribute("applications", service.findAll());
        return "index";
    }
    @GetMapping("/applications/new")
    public String showCreateForm(Model model) {
        Application app = new Application();
        app.setStatus("Applied");
        model.addAttribute("jobApplication", app);
        model.addAttribute("statuses", ApplicationService.ALLOWED_STATUSES);
        model.addAttribute("formTitle", "Add Application");
        model.addAttribute("formAction", "/applications");
        return "application-form";
    }

```

### What's happening:

dashboard puts the sorted list under the name applications, which index.html iterates with th:each. showCreateForm builds an empty Application with a sensible default status, plus the shared statuses list and two strings — formTitle and formAction — that let one template serve both create and edit. Model attributes are the only data channel to Thymeleaf.

### Why it matters:

These GETs never change data. Their job is to prepare exactly the named variables the templates expect; a typo in a name breaks the page, not the database.

---

## Byte 17: Route 3: submitting the create form (POST /applications)

**Builds on:** Bytes 15, 16

### In plain terms:

The browser posts the filled form. The controller validates it, and either re-shows the form with errors or saves and jumps to the new details page.

### The code:

```

[java] (from this project – trimmed to the relevant lines)
@PostMapping("/applications")
public String create(@Valid @ModelAttribute("jobApplication") Application app,
                    BindingResult bindingResult, Model model) {
    applyCustomRules(app, bindingResult);
    if (bindingResult.hasErrors()) {
        model.addAttribute("statuses", ApplicationService.ALLOWED_STATUSES);
        model.addAttribute("formTitle", "Add Application");
        model.addAttribute("formAction", "/applications");
        return "application-form";    // re-show with values + errors
    }
    try {
        Application saved = service.create(app);
        return "redirect:/applications/" + saved.getId();
    } catch (IllegalArgumentException ex) {
        mapServiceError(ex, bindingResult);
    }
}

```

```

        // ... re-add statuses/title/action ...
        return "application-form";
    }
}

```

### What's happening:

@ModelAttribute binds form fields to an Application by matching input names to setters; @Valid runs the @NotBlank/@Size checks and stores failures in BindingResult (which must sit immediately after the model attribute). Custom rules are applied next (Byte 19). On errors the same template is returned with HTTP 200 so entered values and th:errors survive. On success the Post/Redirect/Get pattern issues a redirect to /applications/{id}, preventing duplicate inserts on refresh.

### Why it matters:

This is the write-path template: bind, validate, re-show or redirect. Every form submit in the app follows this shape.

---

### Byte 18: Routes 4-6: detail and edit pages (@PathVariable)

**Builds on:** Byte 15

### In plain terms:

Three routes carry an id in the URL. Two show pages; the third processes the edit form and redirects back to details.

### The code:

```

[java] (from this project – trimmed to the relevant lines)
@GetMapping("/applications/{id}")
public String detail(@PathVariable Long id, Model model) {
    model.addAttribute("jobApplication", service.findByIdOrThrow(id));
    return "application-detail";
}
@GetMapping("/applications/{id}/edit")
public String showEditForm(@PathVariable Long id, Model model) {
    model.addAttribute("jobApplication", service.findByIdOrThrow(id));
    // + statuses, formTitle="Edit Application", formAction="/applications/"+id+"/edit"
    return "application-form";
}
@PostMapping("/applications/{id}/edit")
public String update(@PathVariable Long id,
    @Valid @ModelAttribute("jobApplication") Application app,
    BindingResult bindingResult, Model model) {
    // same validate-then-save-or-reshow shape as create
    service.update(id, app);
    return "redirect:/applications/" + id;
}

```

### What's happening:

@PathVariable extracts {id} from the path and converts it to Long. Detail and edit-form both call findByIdOrThrow, so a bad id never reaches a template — it goes to the 404 handler. The edit form reuses application-form.html; only formTitle/formAction differ, which is how one file serves two modes. Update mirrors create's validation flow but redirects to the same id.

#### Why it matters:

Path variables are the app's addressing scheme. index.html and detail pages build these URLs with real ids (tests assert no /applications/null links remain).

---

#### Byte 19: Route 7: delete is POST-only, never a link

**Builds on:** Byte 15

#### In plain terms:

Deleting uses a small inline form that POSTs. There is deliberately no GET delete route, so crawlers, prefetchers, and refreshes can never wipe data.

#### The code:

```
[java] (from this project – trimmed to the relevant lines)
@PostMapping("/applications/{id}/delete")
public String delete(@PathVariable Long id) {
    service.deleteById(id);
    return "redirect:/";
}
```

#### What's happening:

The detail page renders a <form method="post" action="/applications/{id}/delete"> with a JavaScript confirm() gate. POSTing calls service.deleteById (which 404s first if the id is stale) and redirects to the dashboard. Because browsers only GET plain links, a delete can only happen from that button. No @DeleteMapping exists — HTML forms only support GET/POST.

#### Why it matters:

GET-mutates-data is a classic bug. POST-only plus confirm() is the project's entire CSRF-adjacent safety story (there is no Spring Security in this project).

---

#### Byte 20: Controller validation glue: applyCustomRules + mapServiceError

**Builds on:** Bytes 13, 17

#### In plain terms:

Bean annotations cannot say required-unless-Wishlist, so the controller adds those two checks by hand and pins each failure to the correct form field.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
private void applyCustomRules(Application app, BindingResult r) {
    if (app.getStatus() != null &&
!ApplicationService.ALLOWED_STATUSES.contains(app.getStatus())) {
        r.rejectValue("status", "status.invalid",
            "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected");
    }
    if (app.getStatus() != null && !"Wishlist".equals(app.getStatus())
        && app.getAppliedOn() == null) {
        r.rejectValue("appliedOn", "appliedOn.required",
            "Applied date is required unless status is Wishlist");
    }
}
private void mapServiceError(IllegalArgumentException ex, BindingResult r) {
    if (ex.getMessage() != null && ex.getMessage().startsWith("Status"))
        r.rejectValue("status", "status.invalid", ex.getMessage());
    else r.rejectValue("appliedOn", "appliedOn.required", ex.getMessage());
}
```

### What's happening:

applyCustomRules mirrors service.validate() but at the web layer, using rejectValue(field, code, message) so application-form.html can show the message via th:errors next to status or appliedOn. mapServiceError is the backstop: if the service still throws (e.g. status null, or a race), its message is routed to the matching field by sniffing the prefix Status. Either way the user sees the form again, not a stack trace.

### Why it matters:

Duplicating the rule in two layers is deliberate: the service is the source of truth for data integrity, while the controller copy exists purely to render field-level UX.

---

## Byte 21: Error pages done right: two @ExceptionHandler

**Builds on:** Byte 14

### In plain terms:

Unknown numeric ids and non-numeric ids (like /applications/abc) both land on the same friendly 404 page with a real HTTP 404 status — never the default Whitelabel error.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
@ExceptionHandler(ResourceNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public String handleNotFound(ResourceNotFoundException ex, Model model) {
    model.addAttribute("message", ex.getMessage());
    return "404";
}
```

```

@ExceptionHandler(MethodArgumentTypeMismatchException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public String handleBadId(MethodArgumentTypeMismatchException ex, Model model) {
    model.addAttribute("message", "Application not found: " + ex.getValue());
    return "404";
}

```

#### What's happening:

The first handler catches the service's missing-record signal. The second catches Spring's type-conversion failure when {id} is not a number (abc, null) — which would otherwise be a 400 Whitelabel page. Both set HTTP 404 via `@ResponseStatus` and render `404.html` with a message variable.

#### Why it matters:

Without these, one typo in a URL would show a developer-oriented error page. With them, every bad address degrades to the same branded not-found experience.

---

### Byte 22: Seed data: `DataInitializer` fills an empty table once

**Builds on:** Bytes 9, 2

#### In plain terms:

On first launch the dashboard is not empty: a startup runner inserts three sample applications, but only if the table has zero rows.

#### The code:

```

[java] (from this project – trimmed to the relevant lines)
@Component
public class DataInitializer implements CommandLineRunner {
    @Override
    public void run(String... args) {
        if (repository.count() > 0) return; // never re-seed
        repository.save(sample("Acme Labs", "Python Intern", "Applied",
            LocalDate.of(2026, 9, 16), "", "First application"));
        // + Northstar/Interview, Contoso/Rejected
    }
}

```

#### What's happening:

`@Component` makes Spring pick this up; `CommandLineRunner.run` executes after the context is ready. `count()>0` is the guard — existing user data is never duplicated. `sample()` is a small factory that sets company/role/status/appliedOn/jobUrl/notes; `createdAt` is auto-stamped by `@PrePersist`. Deleting `placement_tracker.db` restores the samples on next boot.

#### Why it matters:

Demos and manual testing need data without manual entry, but re-seeding on every restart would resurrect deleted rows — the count guard is the whole trick.

---

### Byte 23: Frontend shell: base.html fragments (head + nav)

**Builds on:** Byte 15

#### In plain terms:

Every page shares one head and one nav bar through Thymeleaf fragments. No page writes its own `<head>` or navigation from scratch.

#### The code:

```
[html] (from this project – trimmed to the relevant lines)
<!-- base.html -->
<head th:fragment="head(title)">
  <title th:text="${title} + ' | Placement Tracker'">...</title>
  <link rel="stylesheet" th:href="@{/styles.css}" />
</head>
<nav th:fragment="nav" class="nav">
  <a th:href="@{/}" class="brand">Placement Tracker</a>
  <a th:href="@{/applications/new}" class="nav-link">+ Add Application</a>
</nav>
<!-- used as: <head th:replace="~{base :: head('Dashboard')}"> -->
```

#### What's happening:

`th:fragment` names reusable chunks; `th:replace` pulls them into each page with a title parameter. `@{...}` generates context-relative URLs, so links keep working regardless of deployment path. This is the entire frontend-backend URL contract for navigation: `/` and `/applications/new` come from here.

#### Why it matters:

One nav definition means adding a link once updates all five pages, and the stylesheet link here is why `styles.css` applies everywhere.

---

### Byte 24: Frontend dashboard: index.html table, empty state, and links

**Builds on:** Bytes 16, 23

#### In plain terms:

The dashboard is a loop over the applications model attribute. Empty database shows an invitation; otherwise each row shows company, role, status badge, date, and View/Edit links with the real id baked in.

#### The code:

```
[html] (from this project – trimmed to the relevant lines)
<tr th:each="app : ${applications}">
  <td th:text="${app.company}">Acme Labs</td>
  <span th:text="${app.status}"
        th:class="'badge badge-' + ${#strings.toLowerCase(app.status)}">Applied</span>
  <td th:text="${app.appliedOn != null ? app.appliedOn : '-'}">2026-09-16</td>
  <a th:href="@{/applications/{id}(id=${app.id})}" class="btn btn-small">View</a>
  <a th:href="@{/applications/{id}/edit(id=${app.id})}" class="btn btn-small">Edit</a>
</tr>
<div th:if="${#lists.isEmpty(applications)}" class="empty">...No applications yet...</div>
```

### What's happening:

th:each repeats the row per Application; th:text escapes and prints each property. The badge class is computed by lowercasing the status (badge-applied, badge-interview, ...), which styles.css colours. The ternary guards null dates with an em dash. URL templates @ {...} inject app.id — controller tests explicitly assert these contain /applications/1 and never /applications/null.

### Why it matters:

This is the request leg frontend->backend: every View/Edit click is a GET to a controller route from Byte 18, built here with live ids.

---

## Byte 25: Frontend form: ONE template for create AND edit

**Builds on:** Bytes 16, 18, 20

### In plain terms:

application-form.html binds every input to the jobApplication object. The same file renders a blank Add form and a pre-filled Edit form — only the title and post address change.

### The code:

```
[html] (from this project – trimmed to the relevant lines)
<form th:object="${jobApplication}" th:action="@({formAction})" method="post" class="form">
  <input type="text" th:field="*{company}" placeholder="e.g. Acme Labs" />
  <p th:if="${#fields.hasErrors('company')}" th:errors="*{company}" class="error"></p>
  <select th:field="*{status}">
    <option th:each="s : ${statuses}" th:value="${s}" th:text="${s}">Applied</option>
  </select>
  <input type="date" th:field="*{appliedOn}" />
  <!-- + role, jobUrl, notes, Save/Cancel -->
</form>
```

### What's happening:

th:object selects the form-backing bean; th:field generates name/id/value bindings per property (and repopulates user input after a validation failure). th:action reads the controller-supplied formAction (/applications for create, /applications/{id}/edit for edit). th:errors plus #fields.hasErrors display the BindingResult messages from Byte 20 beside the right field. The status <select> is built from the same ALLOWED\_STATUSES list the service validates against.

### Why it matters:

This is the response leg and the next request leg in one: it displays backend state as HTML inputs, and its POST is what Bytes 17-18 consume. Understand `th:object/th:field` and you understand all form data flow.

---

### Byte 26: Frontend detail: `application-detail.html` + POST delete button

**Builds on:** Bytes 18, 19

### In plain terms:

The detail page prints every field of one application and offers three exits: Edit (GET), Delete (POST form with confirmation), and Back to dashboard.

### The code:

```
[html] (from this project – trimmed to the relevant lines)
<h1><span th:text="${jobApplication.company}">Acme</span> – <span
th:text="${jobApplication.role}">Intern</span></h1>
<a th:href="@{/applications/{id}/edit(id=${jobApplication.id})}" class="btn btn-
primary">Edit</a>
<form th:action="@{/applications/{id}/delete(id=${jobApplication.id})}" method="post"
      onsubmit="return confirm('Delete this application?');">
  <button type="submit" class="btn btn-danger">Delete</button>
</form>
```

### What's happening:

All values come from the `jobApplication` model attribute; conditionals show fallbacks (Not set (Wishlist), em dash) for empty `jobUrl/notes/appliedOn`. The delete control is a form, not an `<a>` link, so it issues the POST from Byte 19; the inline `confirm()` is the only JavaScript in the project, purely a safety gate. `jobUrl` renders as a clickable `target=_blank` link when present.

### Why it matters:

This page closes the loop: it is where create/update redirects land, and where the only destructive action originates — correctly as a POST.

---

### Byte 27: Frontend 404: the friendly dead-end

**Builds on:** Byte 21

### In plain terms:

A tiny template turns both kinds of bad id into a calm message and a way home, instead of a stack trace.

### The code:

```
[html] (from this project – trimmed to the relevant lines)
<p class="empty-title">404 – Application not found</p>
<p th:text="${message}">Application not found</p>
<a th:href="@{/}" class="btn btn-primary">Back to dashboard</a>
```

### What's happening:

The message variable is set by whichever `@ExceptionHandler` fired. The page reuses the shared head/nav fragments so even an error looks like part of the app. The only action is back to `/`. `styles.css` is presentational only and carries no backend logic.

### Why it matters:

Error UX is part of the backend contract: HTTP 404 + this body is what users (and tests) observe for every missing or malformed id.

---

## Byte 28: Backend proof 1: service unit tests (Mockito, no real DB)

**Builds on:** Bytes 12, 13

### In plain terms:

Ten service tests fake the repository with Mockito so the real SQLite file is never touched. They prove validation, sorting, copying, and 404 behaviour in isolation.

### The code:

```
[java] (from this project – trimmed to the relevant lines)
@ExtendWith(MockitoExtension.class)
class ApplicationServiceTest {
    @Mock ApplicationRepository repository;
    @InjectMocks ApplicationService service;
    @Test void create_appliedWithoutDate_isRejected() {
        app.setAppliedOn(null);
        assertThrows(IllegalArgumentException.class, () -> service.create(app));
    }
    @Test void create_wishlistWithoutDate_isAllowed() {
        app.setStatus("Wishlist"); app.setAppliedOn(null);
        service.create(app);
        verify(repository).save(app);
    }
}
```

### What's happening:

`@Mock` creates a fake repository; `@InjectMocks` injects it into a real service. `when(...).thenReturn(i -> i.getArgument(0))` echoes the saved object back. Tests assert both branches of the conditional rule (Wishlist-without-date passes, Applied-without-date throws), the status whitelist, Bean Validation via a real Validator (company too short, notes >1000 chars), update field-copying, delete delegation, and sorted `findAll`. 18 tests total across both test classes.

### Why it matters:

These tests are executable documentation of the business rules. If someone weakens `validate()`, these fail before users see bad data.

---

### Byte 29: Backend proof 2: controller tests (MockMvc, no browser)

**Builds on:** Bytes 16, 17, 18, 21

#### In plain terms:

Eleven web-layer tests fake the service and fire simulated HTTP requests at the controller, checking pages, redirects, field errors, and the 404 view.

#### The code:

```
[java] (from this project – trimmed to the relevant lines)
@WebMvcTest(ApplicationController.class)
class ApplicationControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean ApplicationService service;
    @Test void create_blankCompany_showsFormWithErrors() throws Exception {
        mockMvc.perform(post("/applications")
            .param("company", "").param("role", "Python Intern")
            .param("status", "Applied").param("appliedOn", "2026-09-16"))
            .andExpect(status().isOk())
            .andExpect(view().name("application-form"))
            .andExpect(model().attributeHasFieldErrors("jobApplication", "company"));
    }
}
```

#### What's happening:

`@WebMvcTest` loads only the controller slice; `@MockBean` stubs the service. `MockMvc` performs get/post calls and asserts status codes (200 page vs 3xx redirect vs 404), view names, model errors, and even rendered HTML content (contains Acme Labs, real `/applications/1/edit` links, never `/applications/null`). Covered: dashboard, valid-create redirect, blank-company error, missing-date error, detail render, prefilled edit, unknown-id 404, valid-update redirect, delete redirect.

#### Why it matters:

These lock the frontend-backend contract: exact paths, exact template names, and field-error behaviour. A renamed route or broken link fails here first.

---

## PUTTING IT TOGETHER

Type an application into the Thymeleaf form and hit Save: the browser POSTs plain form fields to `ApplicationController`, which binds them to an `Application`, runs `@NotBlank/@Size` plus the hand-written `Wishlist/status` rules, and either re-shows the same form with field errors or calls `ApplicationService`. The service re-validates, then `ApplicationRepository` (with zero SQL written by hand) asks Hibernate — via the SQLite dialect from `application.properties` — to INSERT or UPDATE a

row in placement\_tracker.db, auto-stamping createdAt with @PrePersist. Reads reverse the trip: sorted findAll or findByIdOrThrow feeds index.html, application-detail.html, or the shared application-form.html, while unknown or non-numeric ids are caught by two @ExceptionHandler and rendered as the friendly 404 page. DataInitializer seeds three rows only when the table is empty, and 18 Mockito/MockMvc tests pin the whole loop — routes, validation, sorting, redirects, and real-id links — without ever touching the real database file.